

Scientific Programming and Visualization

(MSDM 5002-Fall 2025)

Instructor: Junwei Liu

Office: 4471 Tel: 2358 7971

Email: liuj@ust.hk

Lec: Fri 07:00pm - 09:50pm

Academic Building 2306

Instructional Assistant

SHING Ming Tony

Email: shingm@ust.hk

Office Hour: Meeting upon appointment

Course description

Brief Information/synopsis:

In this course, the students will learn Python and are required to use Python to solve some scientific and real-life problems.

Meanwhile, the students will learn different packages and tools to deal with different types of data. By doing this, they are expected to know how to choose the suitable tools for different types of problems. In addition, the students will know how to visualize the data generated by scientific computing or the data in daily life in suitable ways.

Key Topics:

- Basic programming skill in Python
- Basic data visualization methods
- Typical packages and tools dealing with data

Intended Learning Outcomes

Upon successful completion of this course, students should be able to:

1. Master Python
2. Collect, clean and organize data in different format
3. Choose suitable methods (tables, figures or animations) to represent data clearly
4. Make animation to show the evolution of data

Assessment Scheme

Assessment	Assessing Course ILOs
5% by attendance	
55% by assignments including quizzes	1-4
40% by the final project	1-4

Learning Resources

A. Adopted References and Textbook:

- <https://www.python.org/doc/>
- <https://leetcode.com/>
- Google
- many other websites

B. Lecture Notes and Course Homepage - <http://canvas.ust.hk>

Rough Course Schedule

- Lecture 01-02: Python basics: strings, numbers, operators, variables, program structure, control flow, data structure, etc.
- Lecture 03: Python: functions, modules and packages
- Lecture 04: Accuracy and Speed (Usage of Array)
- Lecture 05: Simple applications
- Lecture 06-09: Graphics: graphs, scatter plots, density plots, 3D graphics and Animation, etc
- Lecture 10: Web scraping
- Lecture 11-12: Pandas
- Lecture 13 (optional): Overall review of the course + Projects

General principle

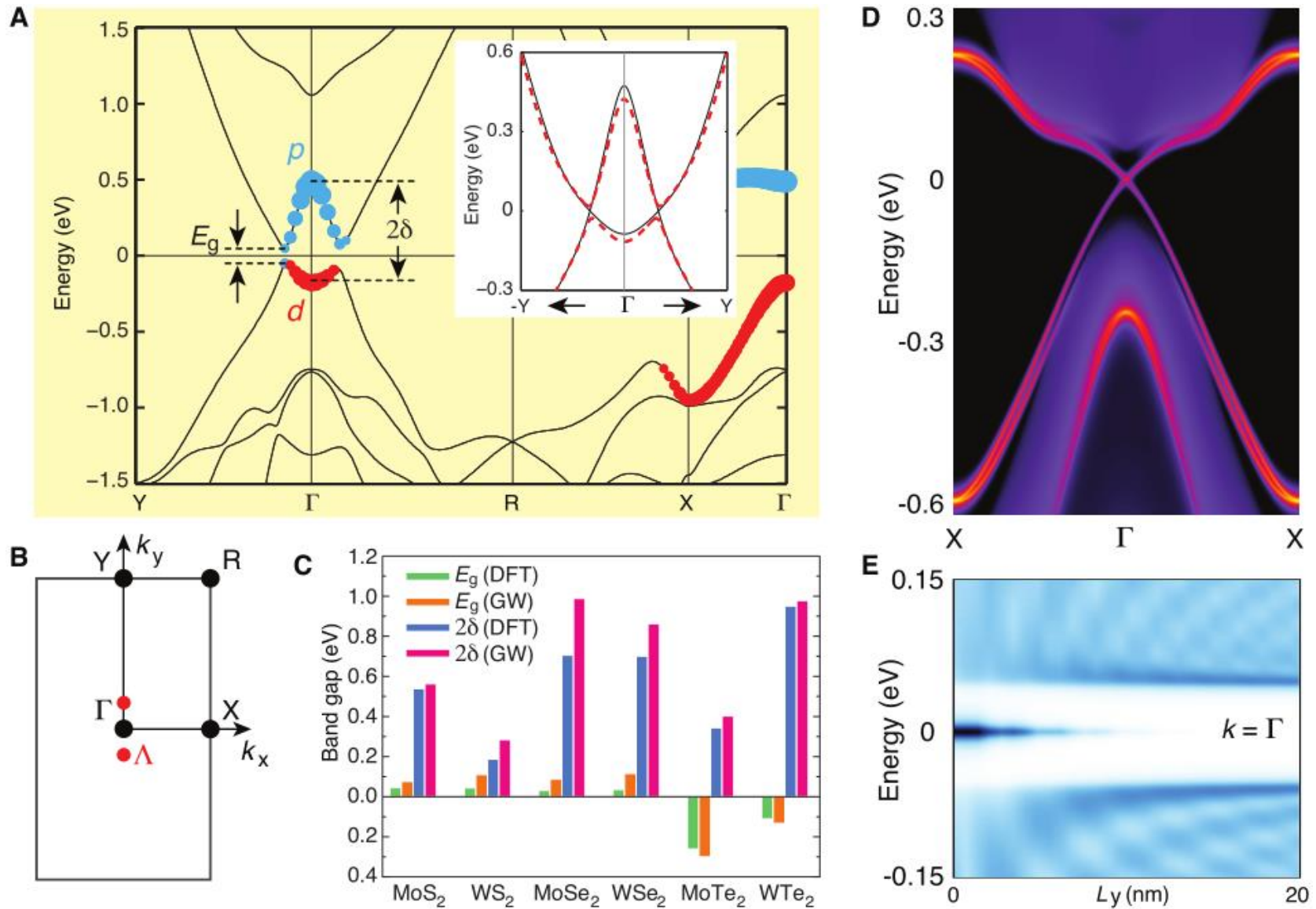
The Simpler The Better

Keep Improving

What is computational science?

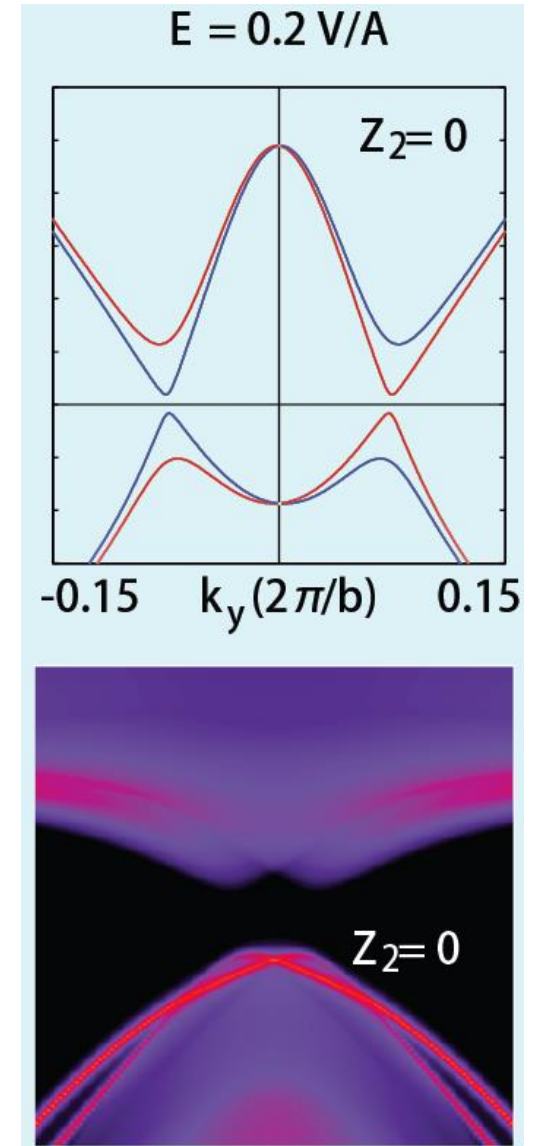
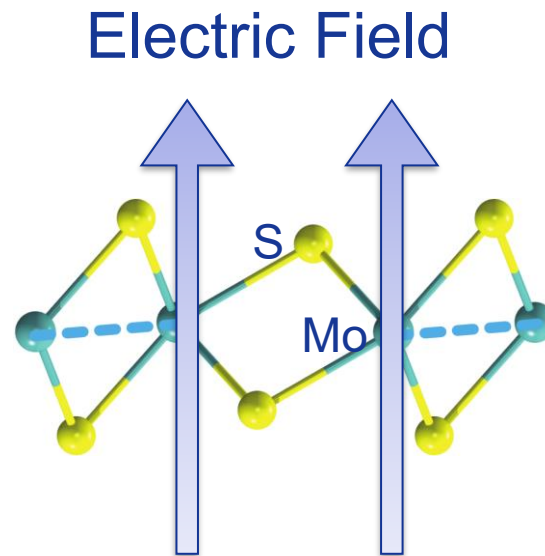
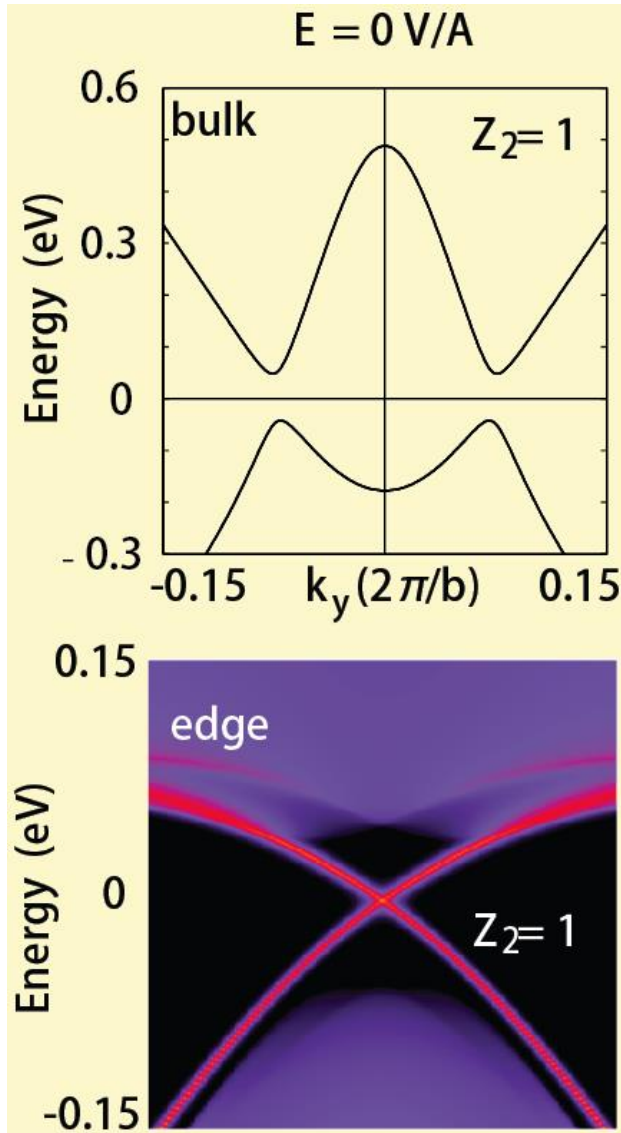
- There are three types of science based on how the problems are solved in general. Experimental, theoretical, and computational.
- Computation is very important. The science we learn before focuses on the fundamental theories, illustrated with examples, whose solution is almost always possible using nothing more than a pen, a sheet of paper. However, this is not how physics is done in real world.
- Most numerical calculations in science fall into one of several general categories, based on the mathematical operations that their solution requires. For example, **integral and derivatives**, **linear algebra tasks** such as matrix inversion or the calculation of eigenvalues, and the solution of **differential equations**, including both ordinary and partial differential equations.

1. Quantum spin Hall effect in 2D materials



X. Qian, J. Liu*, et. al* **Science** 346, 1344 (2014)

Electric tunable - on/off states



Transport experiments

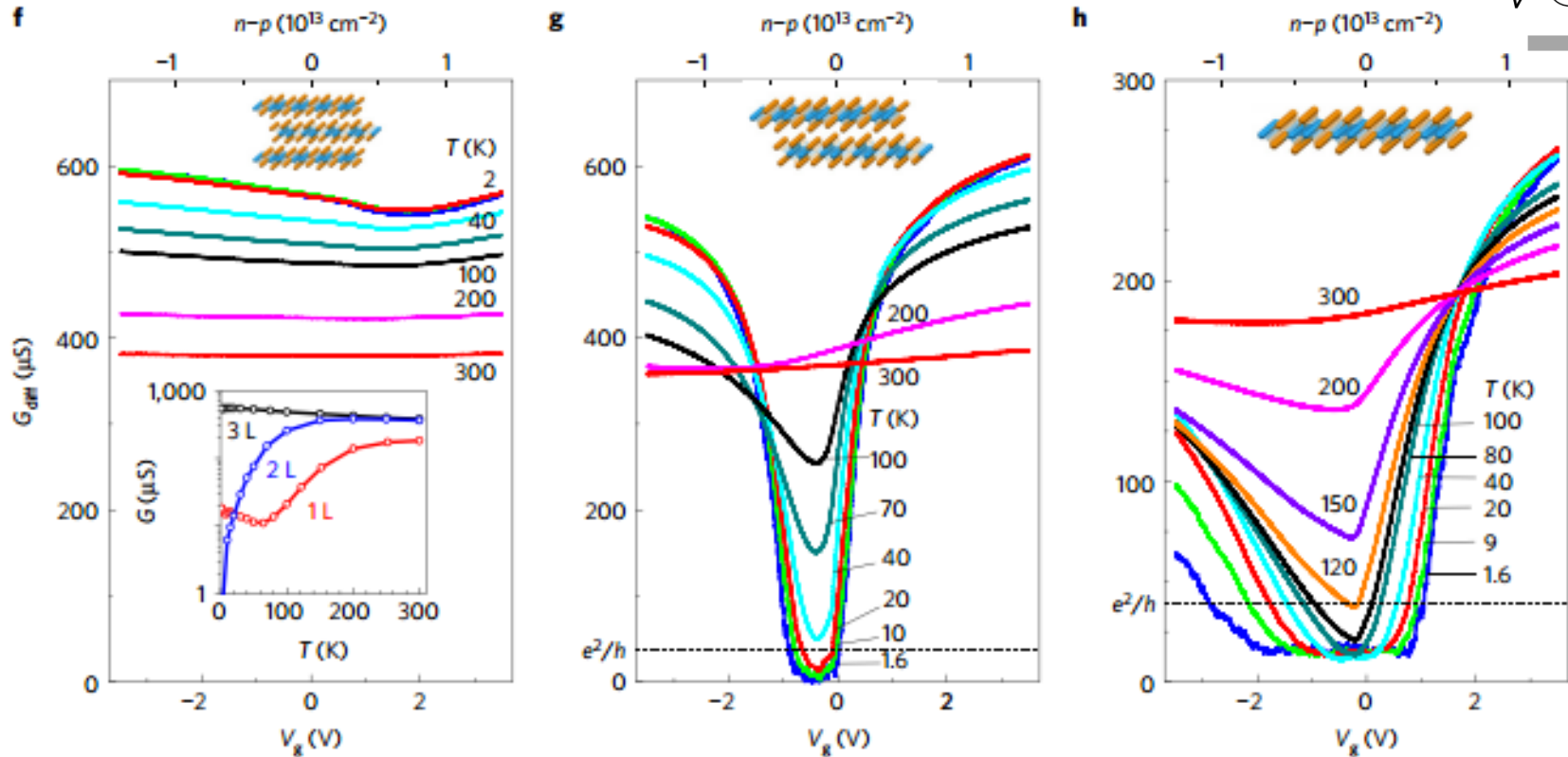
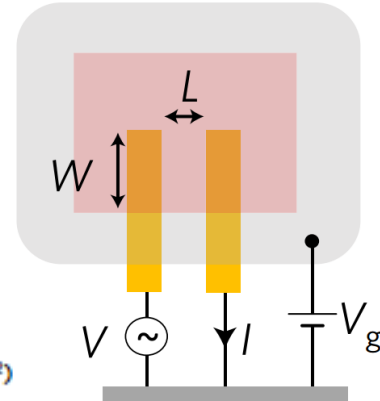
nature
physics

LETTERS

PUBLISHED ONLINE: 10 APRIL 2017 | DOI: 10.1038/NPHYS4091

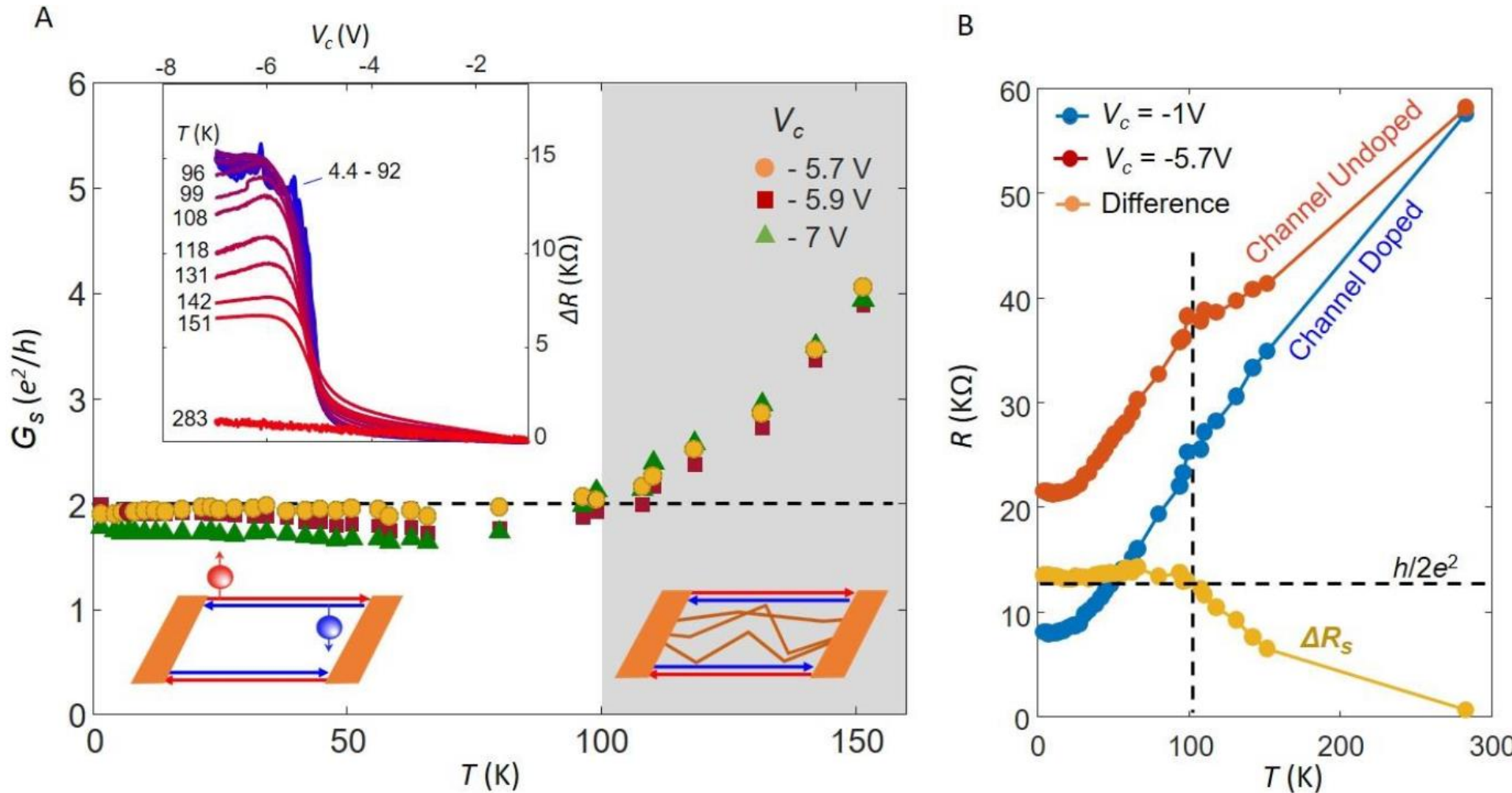
Edge conduction in monolayer WTe₂

Zaiyao Fei¹, Tauno Palomaki¹, Sanfeng Wu¹, Wenjin Zhao¹, Xinghan Cai¹, Bosong Sun¹, Paul Nguyen¹, Joseph Finney¹, Xiaodong Xu^{1,2*} and David H. Cobden^{1*}

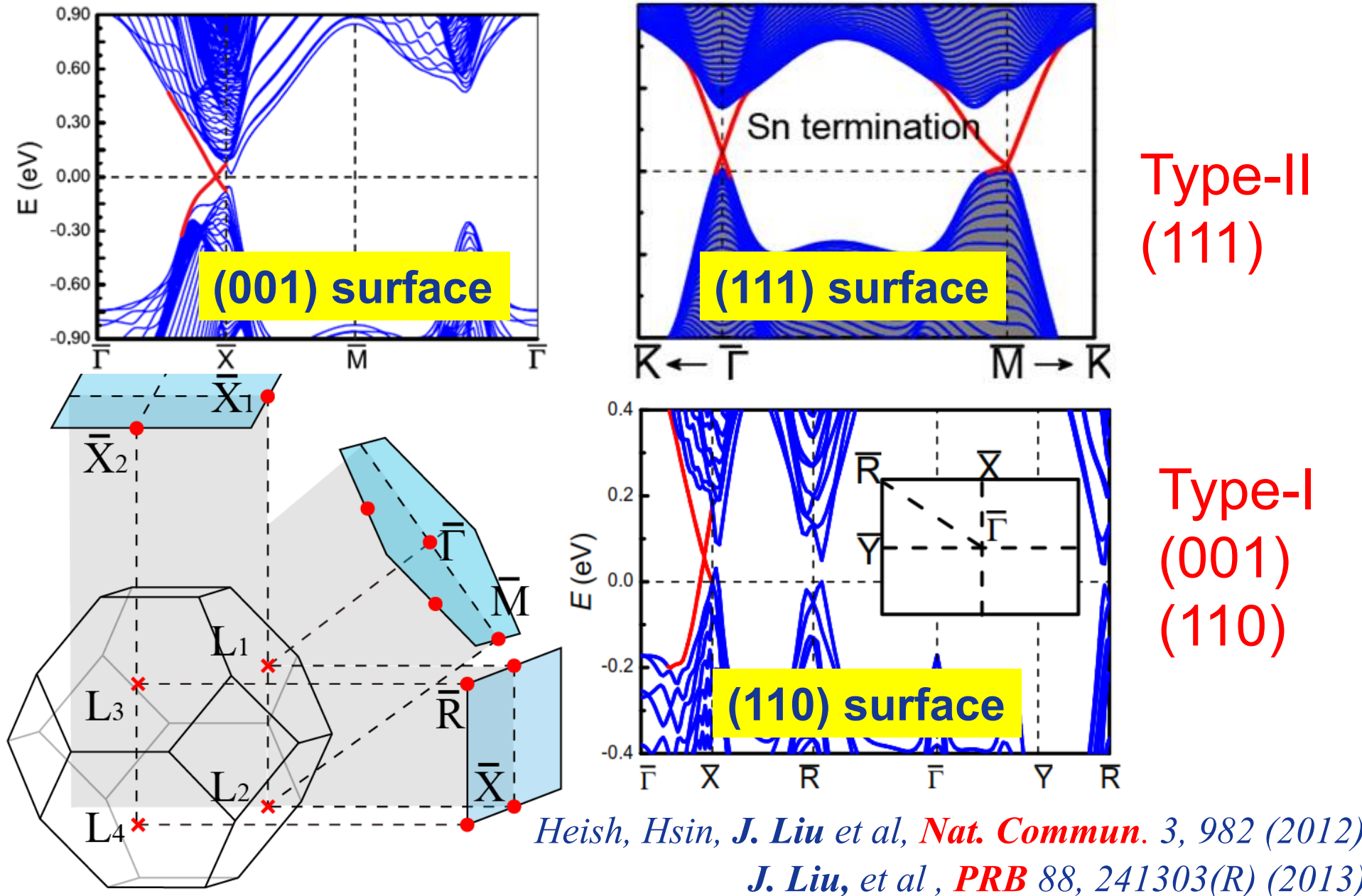


Quantized edge conductance up to 100 K

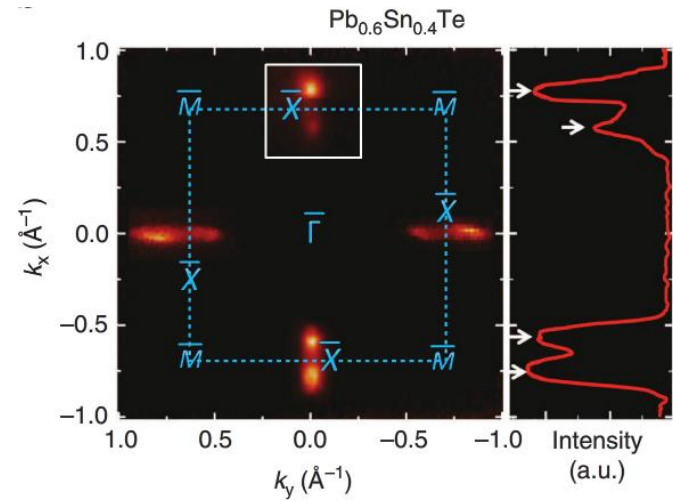
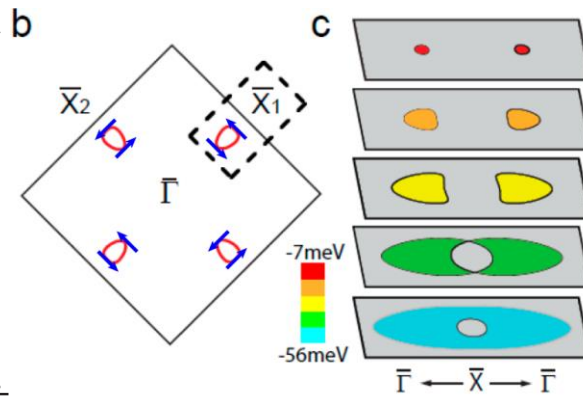
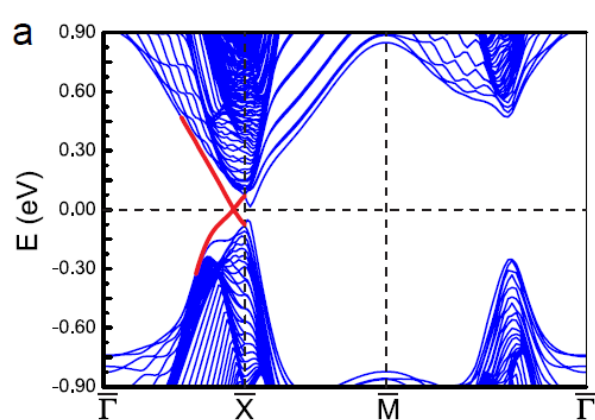
Science



2. Two types of surface states of 3D TCI

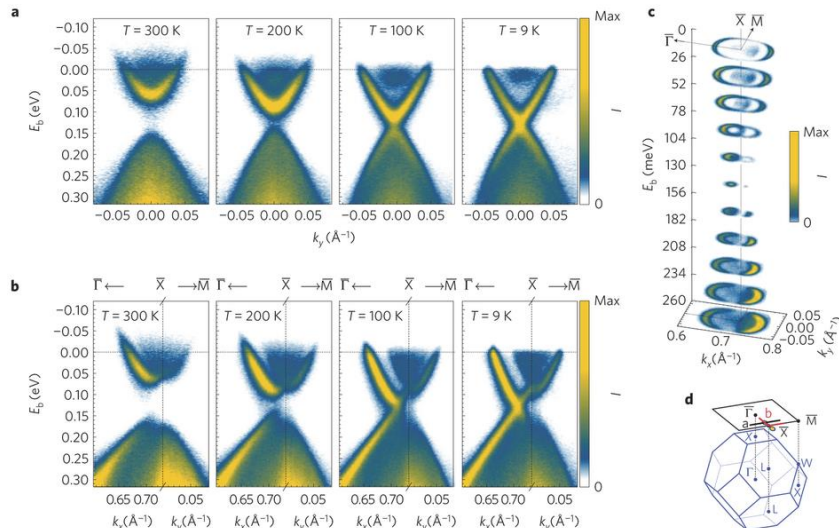


Experiments of $\text{Pb}_{1-x}\text{Sn}_x\text{Te}$ (001) surface

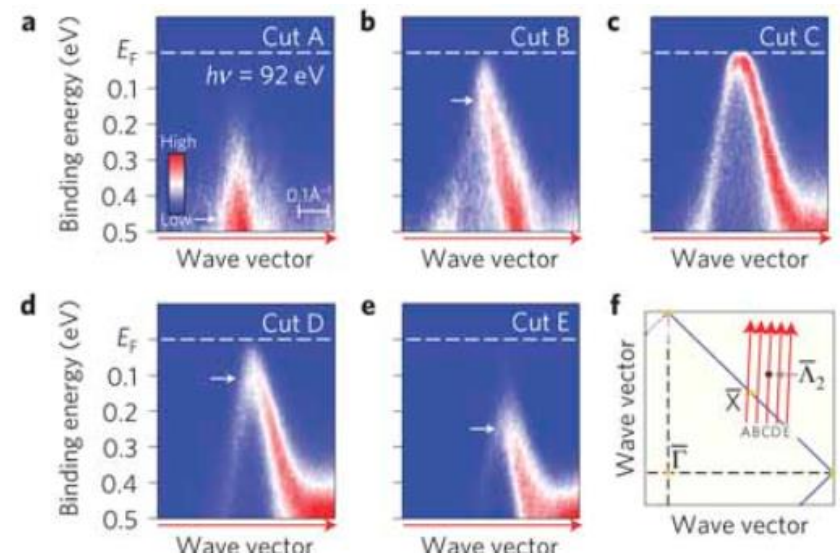


T. Hsieh et al. Nat. Comm. (2012)

S. Xu et al. Nat. Comm. (2012)

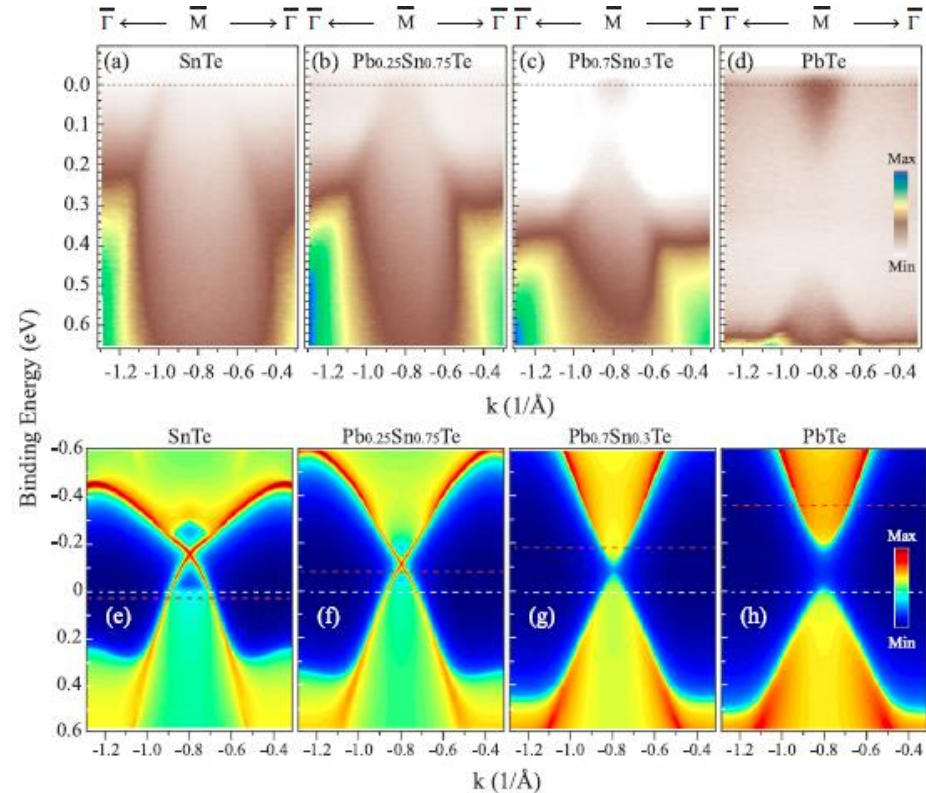
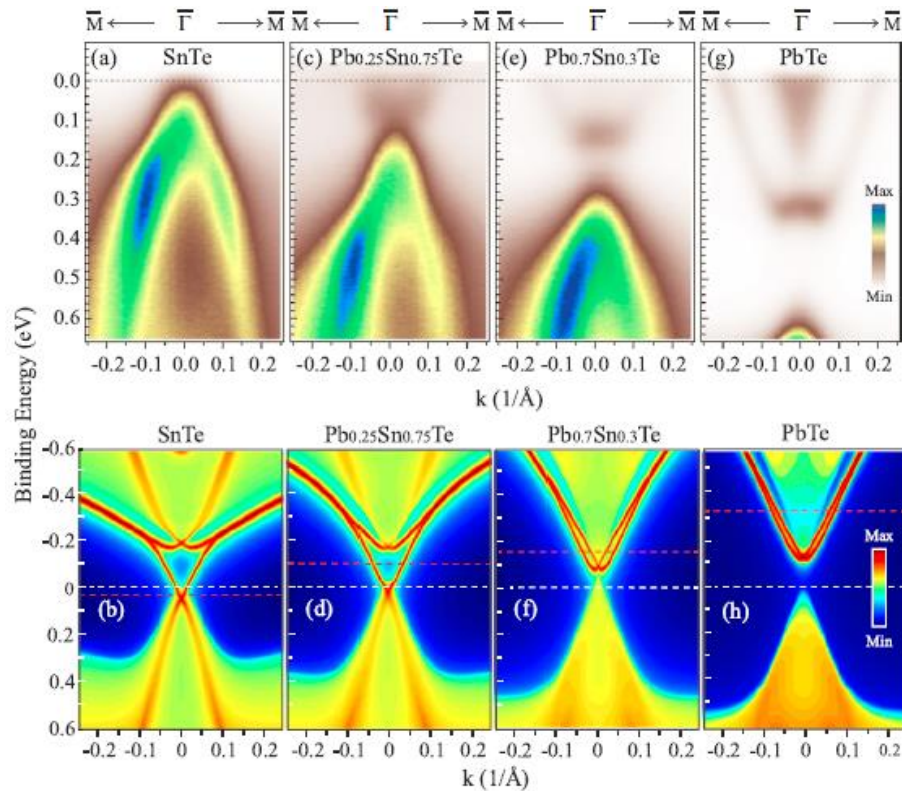


P. Dziawa et al. Nat. Mat. (2012)



Y. Tanaka et al. Nat. Phys. (2012)

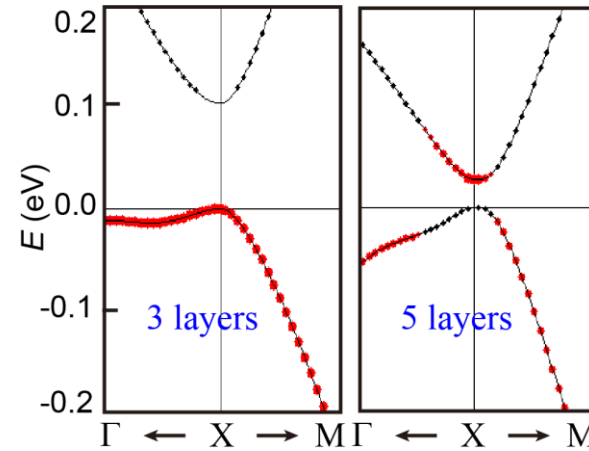
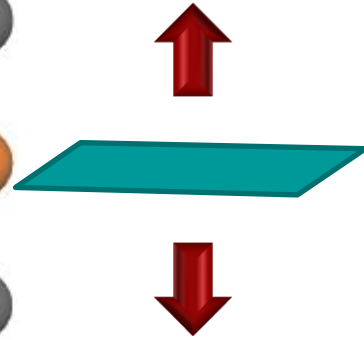
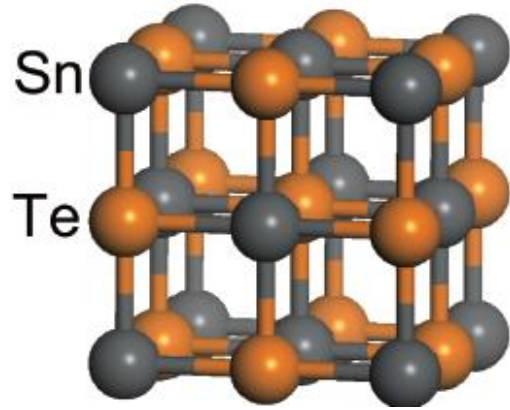
Experiments of $\text{Pb}_{1-x}\text{Sn}_x\text{Te}$ (111) surface



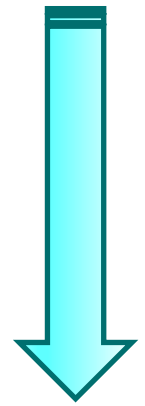
1. Phase transition by varying the ratio of Pb/Sn
2. Dirac cone in both Γ and M points
3. Band bending is important to understand the surface states
4. Te terminated surface

Yan*, J. Liu*, et al, *PRL*. 112, 186801(2014)

3. 2D topological crystalline insulators



**Band
Inversion**



**Non-zero
MCN**

$$H(k) = \tilde{m}\tau_z + (\tilde{v}_x k_x s_x - \tilde{v}_y k_y s_y)\tau_x$$

$$MH(k)M^{-1} = H(k)$$

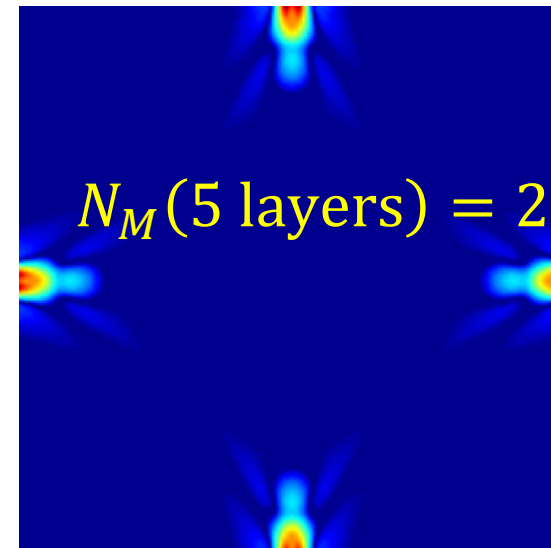
$$M = -is_z\tau_z$$



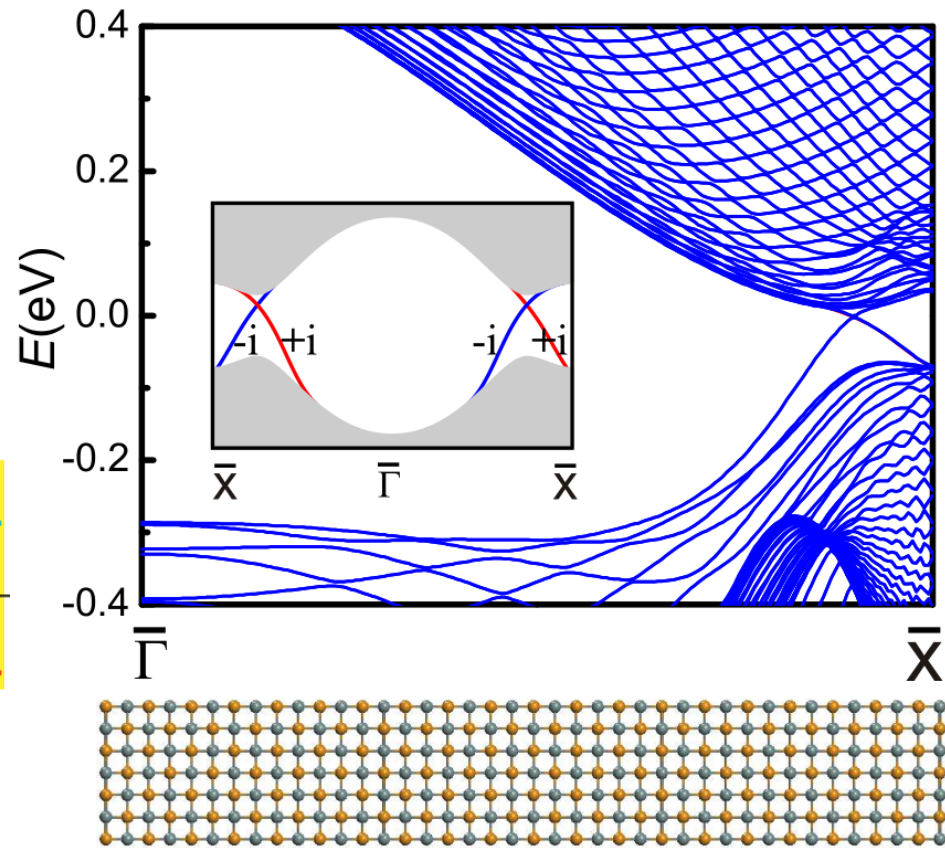
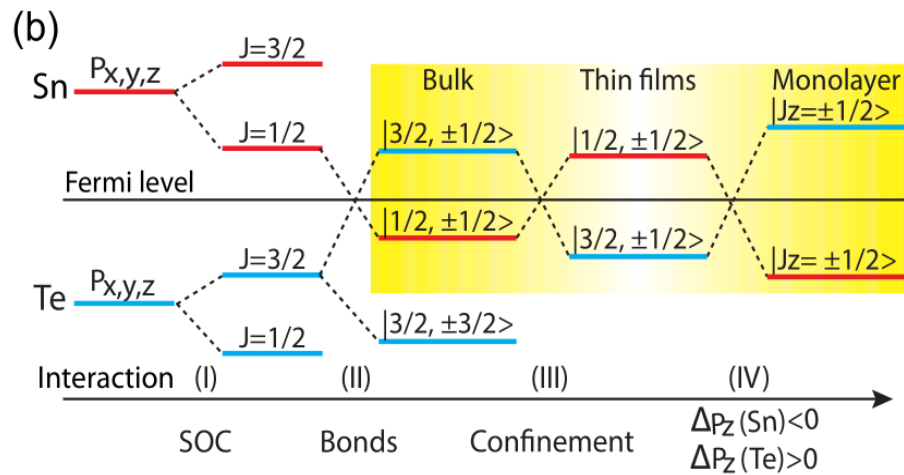
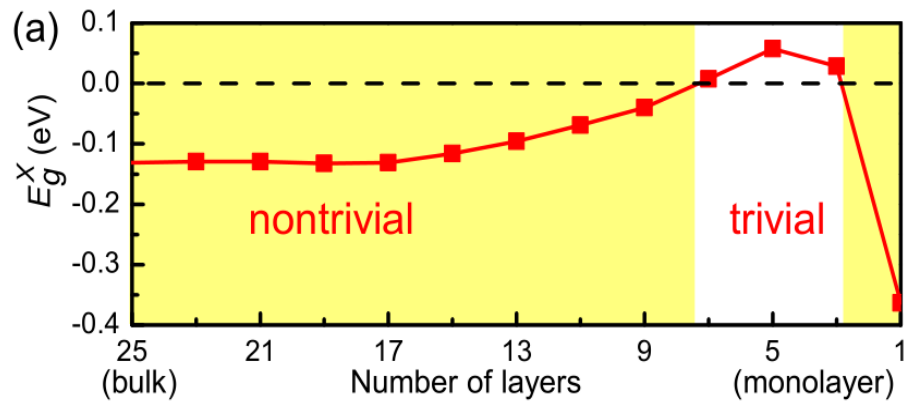
Mirror Chern number

$$N_M = (n_{+i} - n_{-i})/2$$

$$|N_M(\tilde{m} > 0) - N_M(\tilde{m} < 0)| = 2$$



Odd number of layers thin films are 2D TCIs



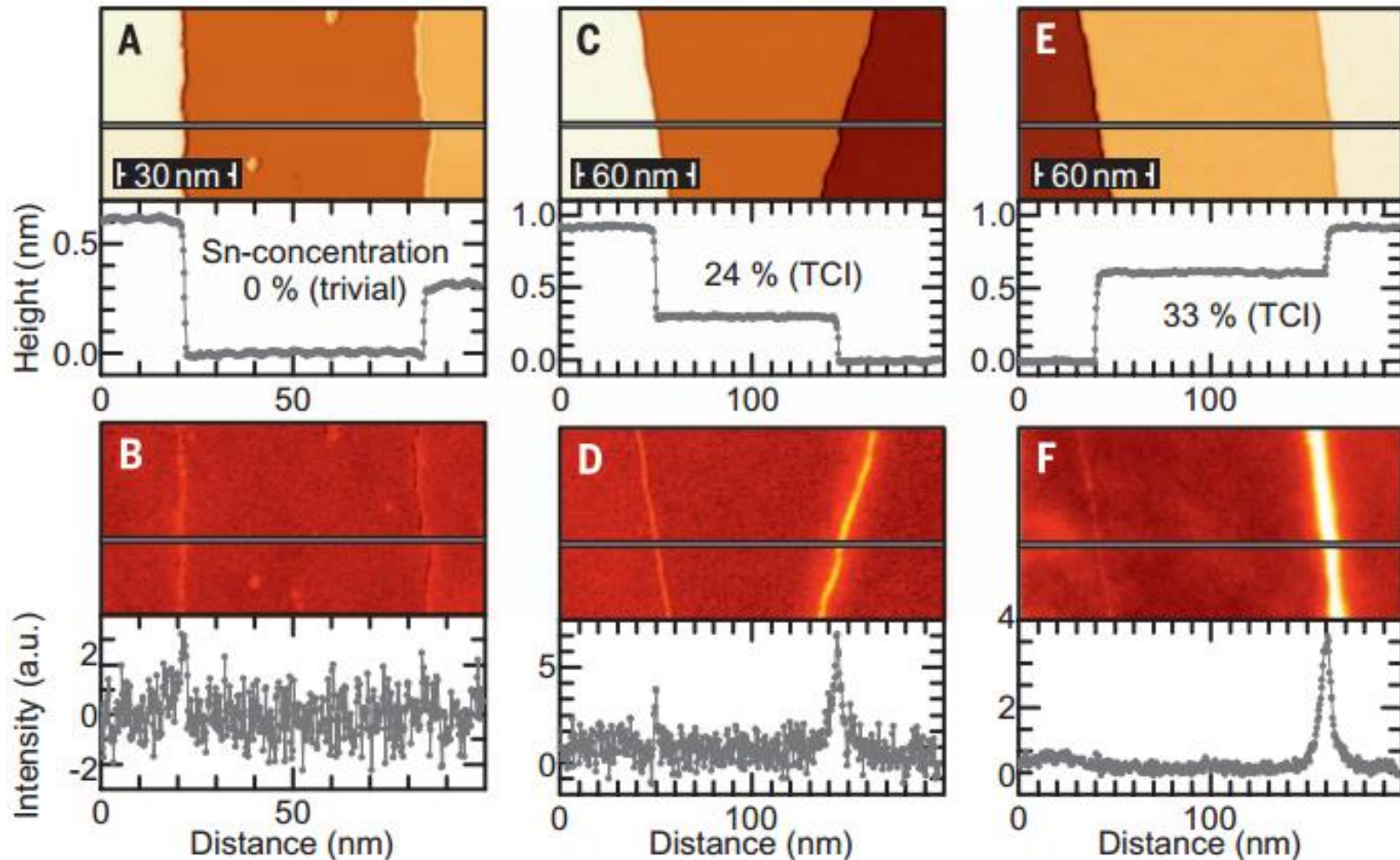
- Odd number of layers thin films can host topological protected edge states
- Even number of layers thin films cannot

Experimental confirmation of our prediction

Science

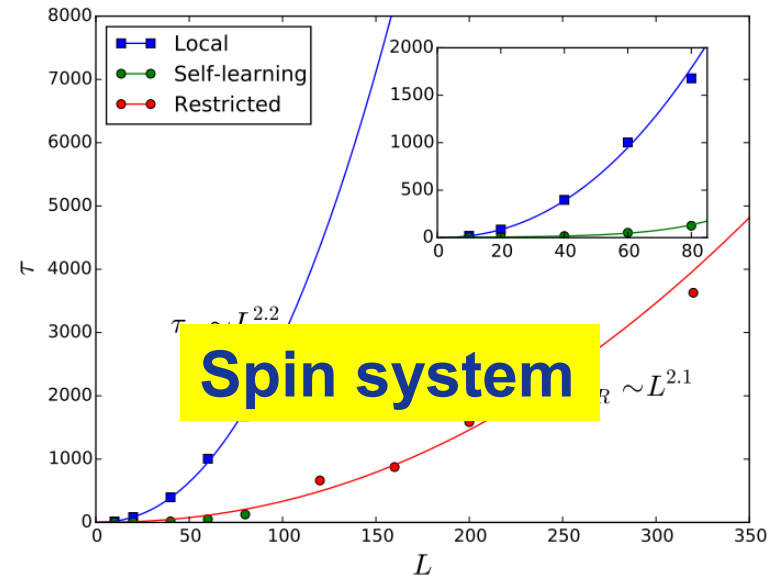
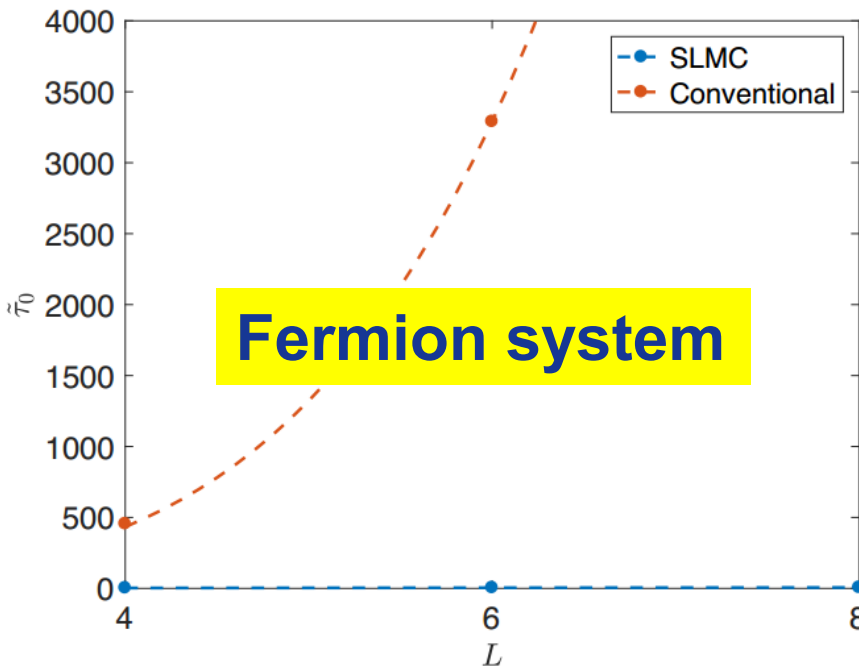
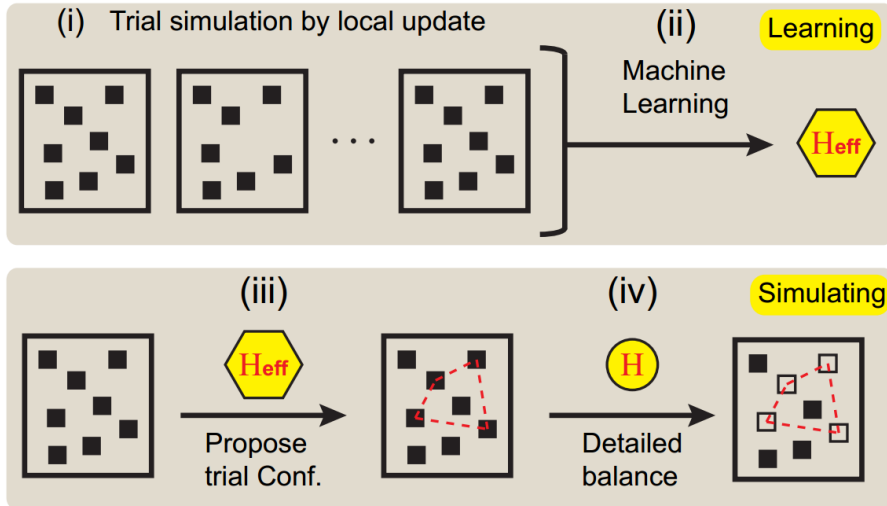
Robust spin-polarized midgap states at step edges of topological crystalline insulators

Paolo Sessi, Domenico Di Sante, Andrzej Szczerbakow, Florian Glott, Stefan Wilfert, Henrik Schmidt, Thomas Bathon, Piotr



P Sessi, et al., Science 354, 1269-1273 (2016)

4. Self-learning Monte Carlo method

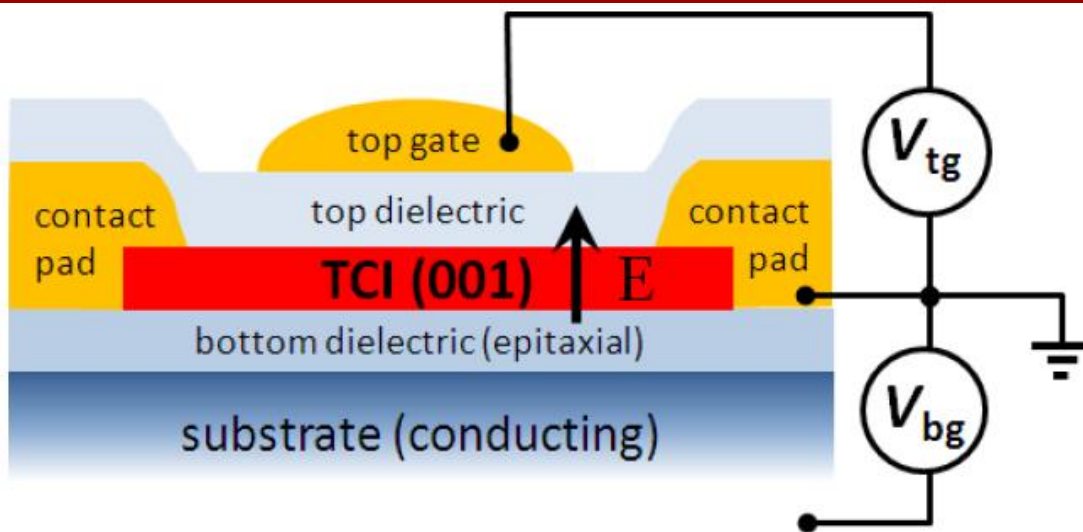


- SLMC can generally speed up Monte Carlo simulations
- SLMC in Fermion systems are more powerful and general, which can easily obtain 1000 times speedup.

J Liu, et al. PRB 95, 041101(R) (2017)

J Liu, et al. PRB 95, 241104(R) (2017)

5. Applications: topological field transistor

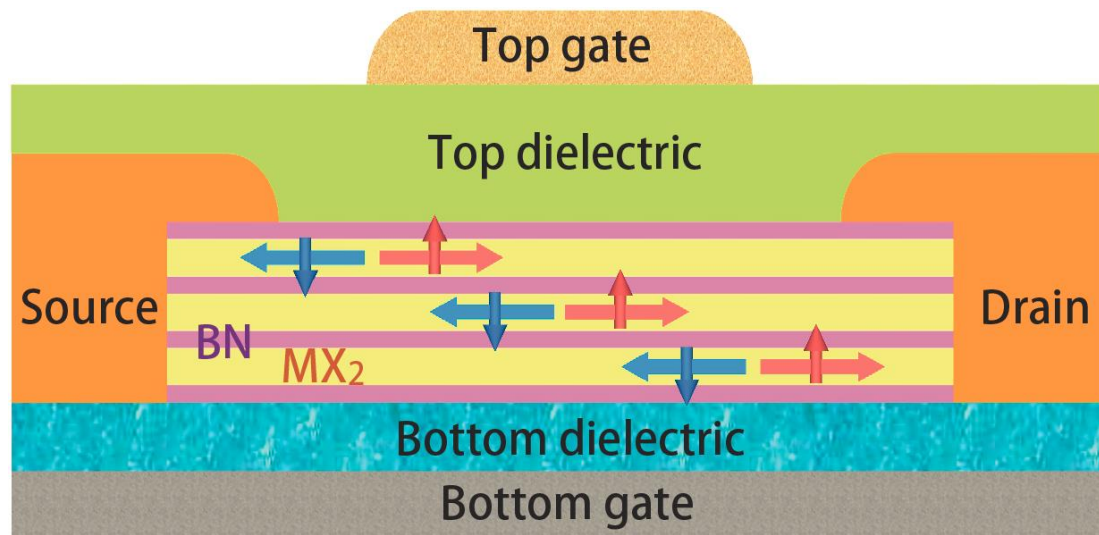


- Based on 2D TCI
- Electric field breaks the mirror symmetry, hence switches ON/OFF states

J. Liu, et al.

Nat. Mater. 13, 178 (2014)

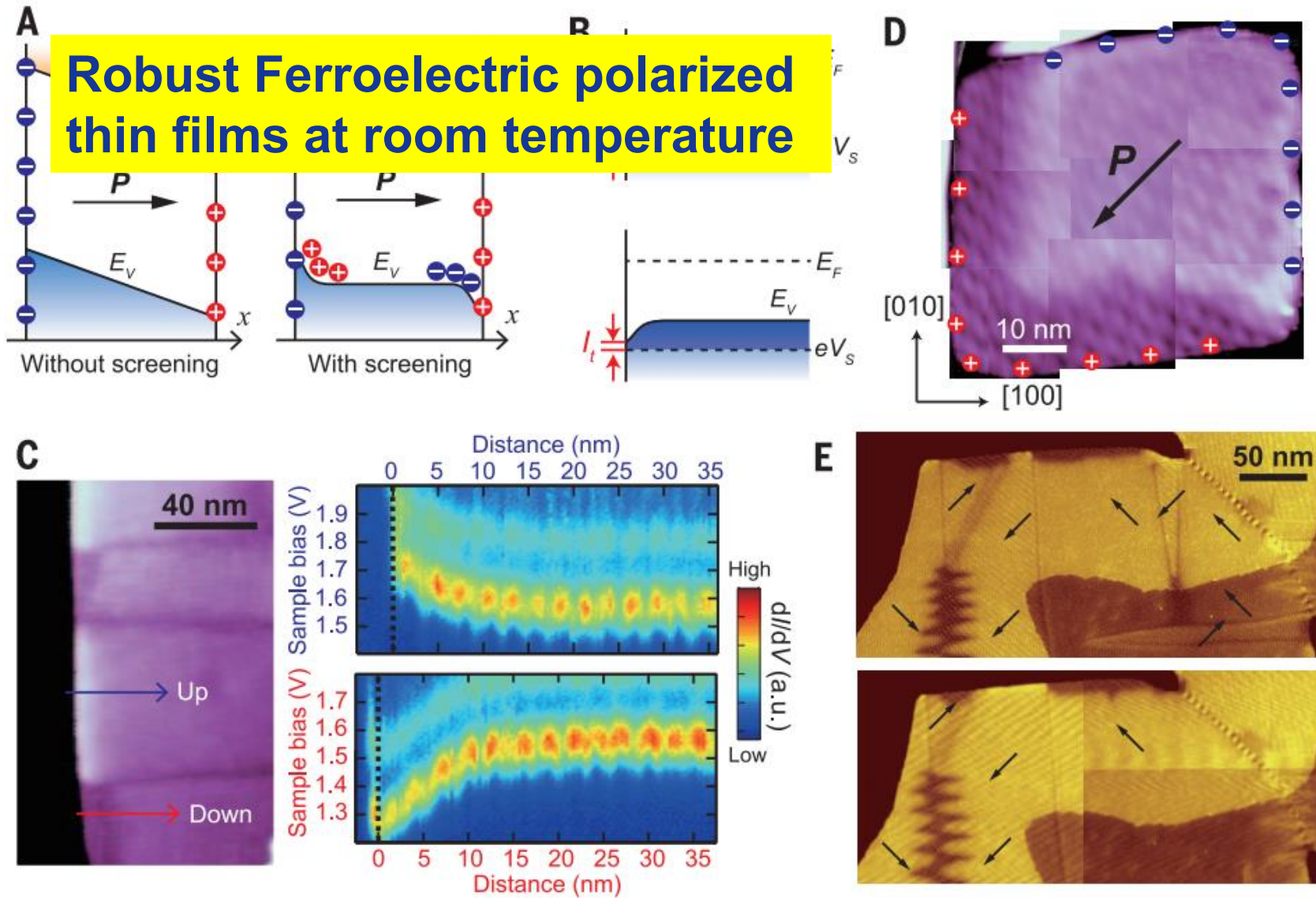
- Based on QSHI
- Electric field induced topological phase transition, hence switches ON/OFF states



X. Qian, J. Liu*, et. al Science* 346, 1344 (2014)

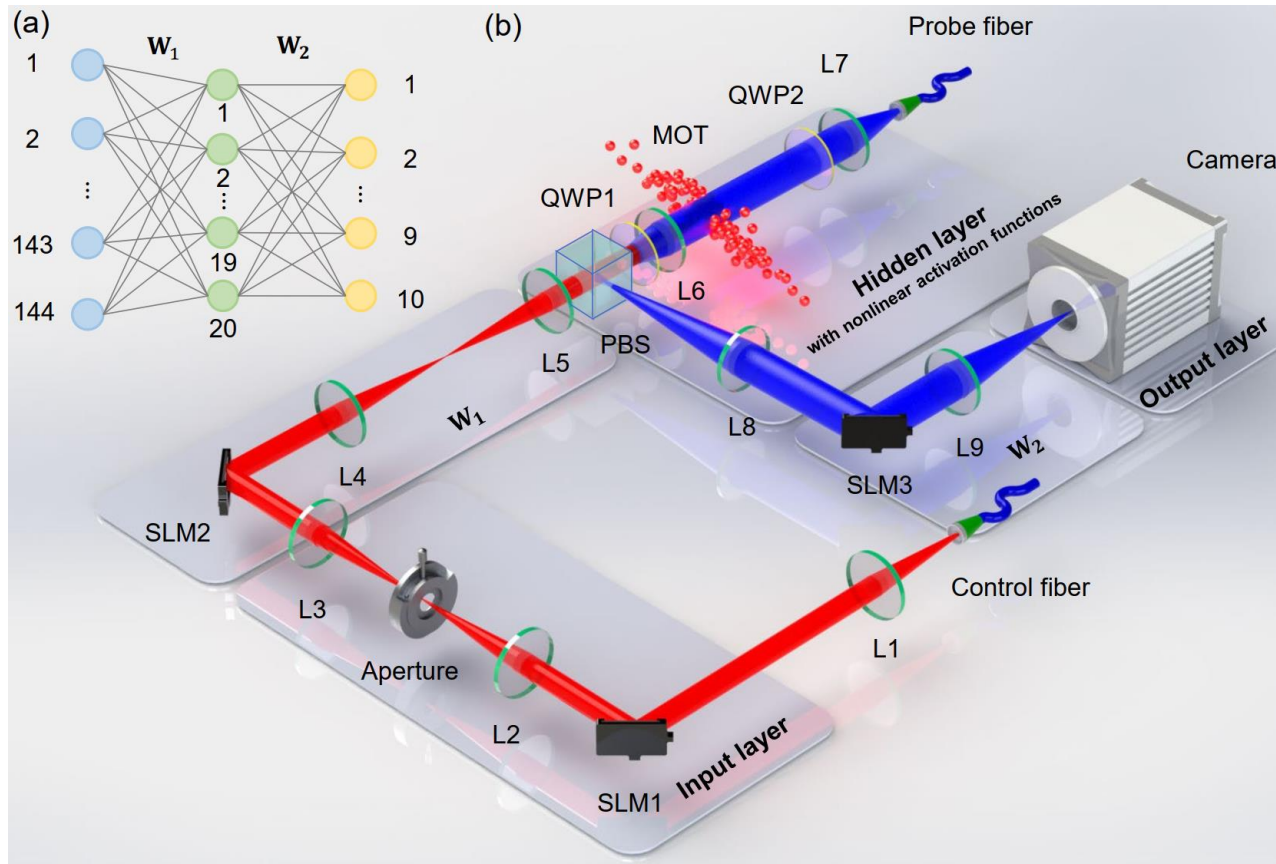
6. Even number of layers thin films of 3D TCI

Robust Ferroelectric polarized thin films at room temperature



K Chang, J Liu*, et al **Science** 353, 274 (2016)*

All-optical deep neural networks



[Optica 6, 1132 \(2019\)](#)

[Phys. Rev. App. 15, 054034 \(2021\)](#)

- At the speed of light
- Infinite parallel calculations
- Reprogrammable
- Scalable

World-first all-optical deep neural network with non-collinear activation functions

What is computer?

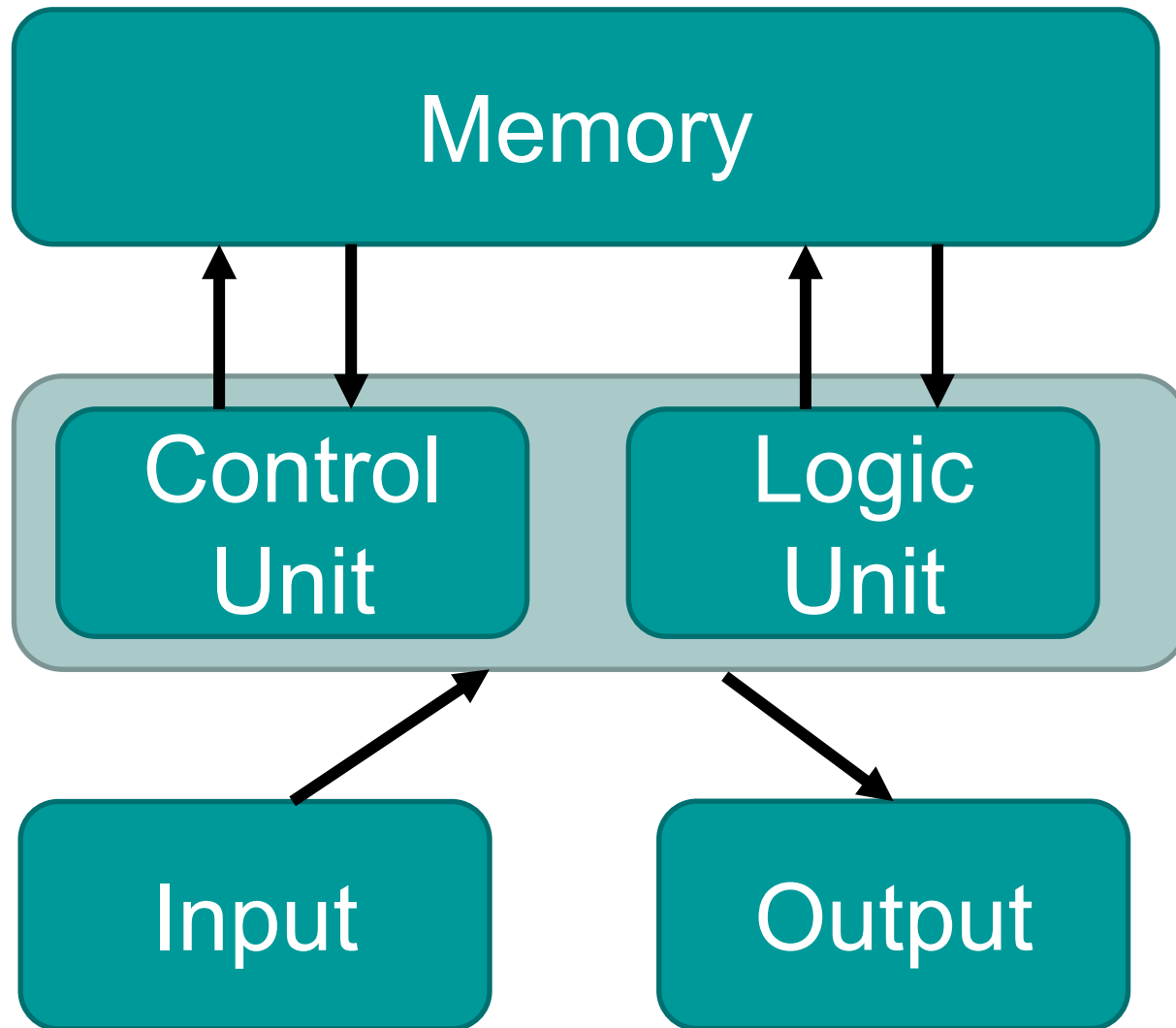
- Computer is really stupid, and it can only do what it is told. And the most important advantage of a computer is that it can do the very tedious repeating operations. It does not feel boring or tired and can give us the reliable results. Such tedious tasks are really challenging for the human.
- On the other hand, Alan Turing demonstrate that computer is really powerful. It can compute anything by using only six primitive operations and a long enough tape: **move left, move right, read, write, scan and do nothing.**
- A programming language is the tool or translator between human and computer. Choose your favorite language, and find the most suitable language for the given problems.

A numerical example

- Square root of a number x is y , $y*y=x$
 - How to get y if I give you the value of x , e.g. $x=17$
- 1) Start with a guess, a
 - 2) If $a*a$ is close enough to x , stop and say a is the answer
 - 3) Otherwise, we make a new guess as $(a+x/a)/2$
 - 4) Repeat 2) and 3), until we get the right answer

a	$a*a$	x/a	$(a+x/a)/2$
3	9	5.6667	4.3333
4.3333	18.7775	3.9231	4.1282
4.1282	17.0420	4.1180	4.1231
4.1231	17.0000	4.1231	4.1231

How does computer work?



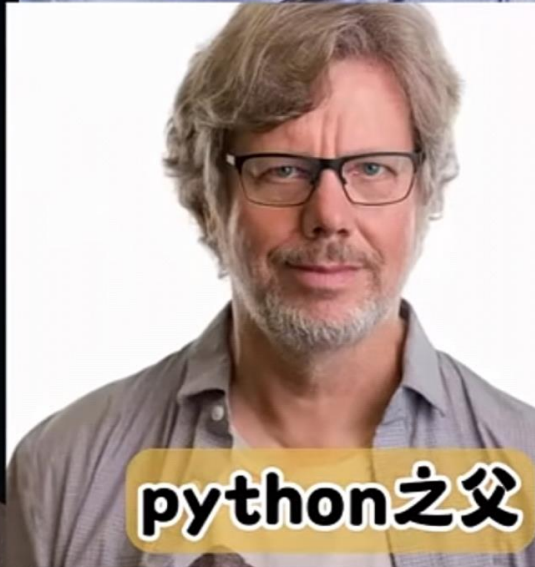
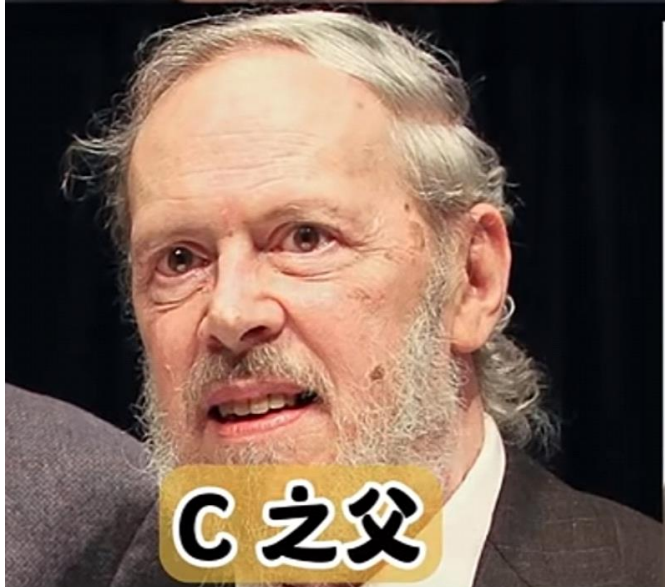
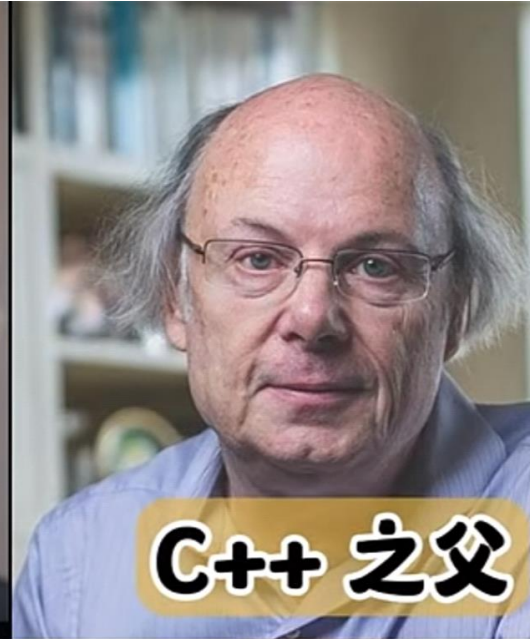
High-Level and Low-Level Languages

- There are two main types of programming languages, high level and low level. Their level depends on how similar the language is to the only language a computer understands, machine code.
- High-level programming languages are ones most developers are familiar with: Java, Ruby, C++, Javascript, etc. A high-level programming language is a language that allows you to tell a computer to do something, but in a syntax that is easy and intuitive for you to understand. It is a totally different language from what a computer understands.
- Low-level programming language is only a slight derivation from machine code. It's much less human readable, yet easier and faster for the computer to understand. It also takes much less memory than a high level language. The most common low level languages are assembly languages.

Compiled and Interpreted Language

S.NO.	COMPILED LANGUAGE	INTERPRETED LANGUAGE
1	A compiled language is a programming language whose implementations are typically compilers and not interpreters.	An interpreted language is a programming language whose implementations execute instructions directly and freely, without previously compiling a program into machine-language instructions.
2	In this language, once the program is compiled it is expressed in the instructions of the target machine.	While in this language, the instructions are not directly executed by the target machine.
3	There are at least two steps to get from source code to execution.	There is only one step to get from source code to execution.
4	In this language, compiled programs run faster than interpreted programs.	While in this language, interpreted programs can be modified while the program is running.
5	In this language, compilation errors prevent the code from compiling.	In this languages, all the debugging occurs at run-time.
6	The code of compiled language can be executed directly by the computer's CPU.	A program written in an interpreted language is not compiled, it is interpreted.
7	This language delivers better performance.	This language example delivers relatively slower performance.
8	Example of compiled language – C, C++, C#, CLEO, COBOL, etc.	Example of Interpreted language – JavaScript, Perl, Python, BASIC, etc.

Why choose Python?



- Free
- Easy
- Powerful

Introduction to Python

1. Basic programming
 - Variables and assignments
 - Variable types
 - Output and input
 - Arithmetic
 - Comments
 - List and arrays
2. Controlling programs
 - If statement
 - While statement
 - For statement
3. Functions, packages, and modules
4. Graphics and visualization
 - Various plots
 - Animation
5. Good programming style

How to use Python?

1. Command line
 2. Directly Run .py file
 3. Jupyter Notebook
 4. Different Integrated Development Environment (IDE)
 - Spyder
 - Pycharm
 - VS Code
 - Visual Studio
 - Others
-
- ✓ Carefully maintain the environment and packages.
 - ✓ Take care of different versions of Python and Packages.

Variables and assignments

```
>>> x = 1
```

This is an *assignment* statement

1. There is a variable called x
2. Assign the value of x to be 1

Name of a variable:

- Can be as long as you like
- Can be any letter or number and underscore symbol “_”
- Cannot start with number
- Cannot contain any other symbols, or spaces
- Upper- and lower-case letters are distinct from one another (x and X are different)
- Give your variable meaningful names that describe what they represent

Variable types

Five types of variable we mainly use:

- Integer: can take integer values and integer values only, both positive and negative values are allowed
- Float: can take real, or floating-point, values. Notice that a floating-point variable can take an integer value, but an integer variable cannot take non-integer values
- Complex: In python the unit imaginary number is called j, not i.
- Bool: True, False
- String: is often used in Python programs but comes up only rarely in physics programming

Why do we need different types of variables?

- Memory space could become a limiting factor in writing the program of many problems. Different type of variables take different size of Memory. (Integer < float < complex)
- Calculations with complex number take longer to complete, because the computer has to calculate both the real and imaginary parts.
- Integer variable are actually more accurate than floating-point variables. Floating-point calculations on computers are not infinitely accurate. The difference between 1 and 0.9999999999999999 or 1.0000000000000001 could be crucially important.
精度很重要 只计算小数点后13位
they are equal in python x == y True but not in the r
- Numerous bugs and problems in computer programs have arisen because of exactly this kind of issue.

How to set the type for a variable?

- The type of variable is set by the value what we give it.
`x=1` and `x=1.0` are different
- We can use functions to set the variable type
`x=float(1)`
- Check the variable type
`type(x)`
- The type of a variable can change as a Python program runs. It is better not to do that although you can. It will make the program more difficult to follow and increase the chance that you may make a mistake in your programming.

string

- Python can also manipulate strings, which can be expressed in several ways. They can be enclosed in single quotes ('...') or double quotes ("...") with the same result [2]. \ can be used to escape quotes:

```
>>> 'it\'s a good example'  
"it's a good example"  
>>> "it's a good example"  
"it's a good example"
```

- Strings can be glued together with the + operator and repeated with *.

```
>>> 'add' + 'up'  
'addup'  
>>> 'times' * 5  
'timestimestimestimestimes'
```

- Two or more string literals (i.e. the ones enclosed between quotes) next to each other are automatically concatenated.

```
>>> 'this' 'is' 'an example'
'this is an example'
```

- Strings can be indexed (subscripted), with the first character having index 0. There is no separate character type; a character is simply a string of size one; Indices may also be negative numbers, to start counting from the right:

```
>>> s='HKUST is a good university'
>>> s[0]
'H'
>>> s[1]
'K'
>>> s[-1]
'y'
>>> s[-2]
't'
```

because X is one variable so it can not change
if x = ['H','K','U','S','T']
x[0] = 'h' is available

```
s = "hello"
# s[0] = 'H'      # 报错: TypeError

# 正确做法, 是创建新字符串
s2 = 'H' + s[1:]  # s2 = "Hello"
```

- The built-in function len() returns the length of a string:

Output statements

```
>>> x=1  
>>> print(x)  
1
```

- `print()` always prints the current value of the variable at the moment the statement is executed.
- `print(x)` and `print('x')` is totally different
- Adding a few words to your program in the output statement can make the output much easier to read and understand.
- It is a dumb but a very useful way to use `print()` to debug your codes.

Some examples

- `\n` means a new line

```
>>> s = 'First line.\nSecond line.'
>>> s
'First line.\nSecond line.'
>>> print(s)
First line.
Second line.
```

- If you don't want characters prefaced by `\` to be interpreted as special characters, you can use raw strings by adding an `r` before the first quote:

```
>>> print(r'First line.\nSecond line.')
First line.\nSecond line.
```

- String literals can span multiple lines. One way is using triple-quotes: `"""..."""` or `'...'`. End of lines are automatically included in the string, but it's possible to prevent this by adding a `\` at the end of the line. The following example:

```
>>> print("""\
... Usage: thingy [OPTIONS]
...     -h                Display this usage message
...     -H hostname       Hostname to connect to
... """)
Usage: thingy [OPTIONS]
    -h                Display this usage message
    -H hostname       Hostname to connect to
```

Format output

- The general form
`print("a = %letter, b = %letter, c = %letter." % (value1, value2, value3))`
- %letter could be %s, %d, %x, %o, %f, %e, %g, %r

% <i>m</i> s	String
% <i>m</i> d	Integer number
% <i>m</i> x, % <i>m</i> X	hexadecimal number
% <i>m</i> o, % <i>m</i> O	Octonary number
% <i>m.n</i> f	Float number
% <i>m.n</i> e	Scientific notation
%g	Automatically choose suitable type %f or %e
%r	The original format

m, n are integers

f-strings in Python

- Python offers a powerful feature called **f-strings** (formatted string literals) to simplify string formatting and interpolation. f-strings is introduced in Python 3.6 it provides a concise and intuitive way to embed expressions and variables directly into strings. The idea is to make string interpolation simpler.
- You can also use f-strings in input()

```
a=2.0
b=10
c='string'
print(f'{a:<20.2f}: {b:>10}:')
print(f'{a:>20.2f}: {c:<10}:')
print(f'{a:^20.2f}:')
print(f'{a:T^20.2f}:')
print(f'{a:_^20.2f}:')
print(f'Test an interesting usage: {a + b = }')
```

```
2.00                :                10:
                    2.00: string      :
                2.00                :
TTTTTTTTT2.00TTTTTTTT:
-----2.00-----:
Test an interesting usage: a + b = 12.0
```


Input statements

```
>>> x = input( "Enter the value of x:" )  
Enter the value of x:
```

- The computer will wait until you type a value on the keyboard.
- The value entered is always interpreted as a string value, even if you type in a number. The computer does not care whether you enter digits, letter, a complete work, or several words.
- We need to convert a string into a number if you want to input a number.

```
>>> x = float(input( "Enter the value of x:" ))  
Enter the value of x:
```

Arithmetic

- $x+y$ addition
- $x-y$ subtraction
- $x*y$ multiplication
- x/y division
- $x**y$ raising x to the power of y
- $x//y$ the integer part of x divided by y
- $x\%y$ modulo, the remainder after x is divided by y
- The end result is the same type as the starting values, or the more general type if there are two different starting types
- Number can multiply string, which mean repeat the string several times.
- It is not true for the matrix

在别的language里面想要交换xy需要建立新的变量
但是在python只需要 `x, y = y, x`

More tricks

- Python modifiers
 $x+=1$, $x-=4$, $x*=3.5$, $x/=5*y$
- The ability to assign the values of two variables with a single statement.
 $x, y = 1, 2.5$ equivalent to $x=1$ and $y=2.5$

how about this one, $x, y = 2*z+1, (x+y)/3$
- We do not need an additional temporary variable to swap the values of x and y
 $x, y = y, x$

Controlling programs: if statement

- ***if conditions :***
codes
- ***if conditions :***
codes
else :
codes
- ***if conditions :***
codes
elif conditions :
codes
else :
codes
- logical operators
==, >, >=, <, <=, !=
- x==1 is different form x=1
- Combine different conditions
or, and
- The indentation is crucial in Python, spaces at the beginning of lines do have an effect with an ***if*** statement or other controlling statement.

Controlling programs: while statement

- ***while conditions :***
codes

- ***while conditions :***
codes
else :
codes

- ***while conditions :***
codes
if conditions:
codes
break or continue
codes

- check if the condition is met. If it is, it will executes the indented block of code immediately following; if not, it skips the block.
- If the condition is met and the block is executed, the program then loops back to the beginning and checks the condition again.
- Used to ensure some conditions is met in a program or to keep on performing an operation until a certain situation is reached.
- Infinite loops

Combine different conditions

- In many cases, we need to combine different conditions to get more complicated condition in if and while statement
- Three type of operators are “and”, “or”, “not” and has first piority must do all and then continue to or
- If there are more than two operators, the rules are as following
 1. From left to right
 2. The priority of “and” is higher than “or”
- The return is not always a bool type
- Check it carefully before use it

Examples and Practices

1. a= True or False and True and False
2. a= (True or False) and True and False
3. a= 1 and 2
4. a= 2 and 1
5. a= 1 or 2
6. a= 2 or 1
7. a= not 1
8. a= not 0
9. a= 0 and True; b = 0 or True
10. a= 0 and False; b = 0 or False
11. a= 1 and True; b = 1 or True
12. a= 1 and False; b = 1 or False
13. a= 2 and True; b = 2 or True ₂
14. a= 2 and False; b = 2 or False

type(a); type(b); print(a); print(b)

Controlling programs: for statement

- We can use while statement to repeatedly execute a given section of code. However, it is rarely used in practice. Instead, there is another much more commonly used loop construction in the Python, **for** loop.

check continues carefully it not same as break

- A for loop is a loop that runs through the elements of a list or array in turn.

```
r = [ 1, 3, 7, 8]
for n in r:
    print(n)
    print(2*n)
print("Finished")
```

- Use function **range(a,b, steps)**

```
for n in range(5)
    print(n**2)
```

Comments

- Like other computer languages, Python also have the important feature, ***comments***
- In Python, any program line that starts with a hash mark “#” is ignored completely by the computer.
- Comments make no difference to the way a program runs, but they do make huge difference for you!
- Comments do not have to start at the beginning of a line. Python ignores any portion of a line that follows a hash mark, whether the hash mark is at the beginning or not.

Example I

- The problem: a ball is dropped from a tower of height h . It has initial velocity zero and accelerates downwards under gravity. The challenge is to write a program that asks the user to enter the height h in meters of the tower and a time interval t in seconds, then prints on the screen the height of the ball above the ground at time t after it is dropped, ignoring air resistance.

```
h=10
```

```
t=1
```

```
s=h-9.81*t**2/2
```

```
print("\nThe initial height of the ball is ", h, " meters")
```

```
print("After", t, " seconds, the height is s=", s, " meters")
```

- You need to change your code every time you change the parameters

```
h=float(input("Enter the height:"))
```

```
t=float(input("Enter the time:"))
```

```
s=h-9.81*t**2/2
```

```
print("\nThe initial height of the ball is ", h, " meters")
```

```
print("After", t, " seconds, the height is ", s, " meters")
```

- What if $h < 0$, $t < 0$ or $s < 0$? It does not make any sense!

A ball dropped from a tower with control

```
h=float(input("Enter the height:"))
t=float(input("Enter the time:"))

while h<0:
    print("You need to input a positive number")
    h=float(input("Enter the height:"))

while t<0:
    print("You need to input a positive number")
    t=float(input("Enter the time:"))

s=h-9.81*t**2/2

print("\nThe initial height of the ball is ", h, " meters")
if s<0:
    print("Before", t, " seconds, the ball has already hitted the groud.")
else:
    print("After", t, " seconds, the height is ",s, " meters")
```

- The program becomes more reasonable with the controlling statements and less unexpected (wrong) results
- Is it good enough?